

GoF Design Patterns - Detailed Quick Reference

Creational Patterns

Factory Method

Intent: Creates objects through an interface while allowing subclasses to decide which concrete class to instantiate.

When to use: When object creation depends on runtime conditions.

Benefits: Decouples creation from use.

Abstract Factory

Intent: Creates families of related objects without specifying concrete classes.

When to use: Cross-platform UI, database drivers.

Benefits: Keeps related products consistent.

Builder

Intent: Builds complex objects step by step.

When to use: Complex configuration objects.

Benefits: Readable construction.

Prototype

Intent: Creates new objects by cloning existing ones.

When to use: Expensive object initialization.

Benefits: Fast copying.

Singleton

Intent: Ensures only one instance exists.

When to use: Configuration, logging.

Benefits: Global controlled access.

Structural Patterns

Adapter

Intent: Makes incompatible interfaces work together.

When to use: Legacy integration.

Benefits: Reuse existing code.

Bridge

Intent: Separates abstraction from implementation.

When to use: Multiple dimensions of variation.

Benefits: Independent evolution.

Composite

Intent: Treats individual and grouped objects uniformly.

When to use: File systems, UI trees.

Benefits: Recursive structures.

Decorator

Intent: Adds behavior dynamically.

When to use: Streams, middleware.

Benefits: Open/Closed principle.

Facade

Intent: Provides simplified interface.

When to use: Complex libraries.

Benefits: Cleaner APIs.

Flyweight

Intent: Shares common state.

When to use: Text editors, game tiles.

Benefits: Memory savings.

Proxy

Intent: Controls access to another object.

When to use: Lazy loading, caching.

Benefits: Security and performance.

Behavioral Patterns

Chain of Responsibility

Intent: Passes requests through handlers until one handles it.

When to use: HTTP middleware.

Benefits: Loose coupling.

Command

Intent: Encapsulates requests as objects.

When to use: Undo/redo, queues.

Benefits: Request history.

Iterator

Intent: Sequential traversal without exposing structure.

When to use: Collections.

Benefits: Consistent iteration.

Mediator

Intent: Centralizes communication.

When to use: Chat rooms, dialogs.

Benefits: Reduces dependencies.

Memento

Intent: Captures and restores state.

When to use: Undo features.

Benefits: State recovery.

Observer

Intent: Publishes changes to subscribers.

When to use: Events.

Benefits: Loose event-driven design.

State

Intent: Changes behavior based on state.

When to use: Order workflow.

Benefits: Removes complex conditionals.

Strategy

Intent: Swaps algorithms at runtime.

When to use: Sorting, payment methods.

Benefits: Easy extensibility.

Template Method

Intent: Defines algorithm skeleton with customizable steps.

When to use: Frameworks.

Benefits: Code reuse.

Visitor

Intent: Adds new operations without modifying classes.

When to use: AST processing.

Benefits: Open for operations.